A3 · 学生实操手册 —— 从零补 Swing + 客户端 Socket,SOLO 生成读者端 + 借书事务审查

> **对应**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元③ **用法**:跟着一步一步做,每一步都有【操作】【✅ 怎么算做好】【卡住怎么办】。讲师带做时你同步做,课后自己做也照这份来。 **前置**:单元② 的**服务端能启动**(监听 9000);MySQL 里 `library_cs` 库有种子数据(admin/123456、zhangsan/123456、lisi/123456);工作区里 `.trae/rules`(单元① 的 PRD)在。 **学情提示**:今天要用到的 **Swing 图形界面** 和 **客户端 Socket** 你还没在课上学过。**动手前先看下面"零基础速记"两节**(或母本 Part 1–2 / 讲义二~五节),看懂再跟做。看不懂的地方,当场让 AI 用中文逐行解释。

今天要做什么(先读这一段)

单元① 写好了图纸(`.trae/rules`),单元② 把服务端搭起来、把"登录"这条 C/S 链路打通了(客户端发 `LOGIN|admin|123456` → 服务端查库 → 返 `OK|admin|系统管理员|1`)。但那时候还没有真正的界面。

今天做客户端,并且第一次让**真界面**连上服务端。但客户端有两块你还没学过:**Swing 图形界面**(窗口、表格、按钮怎么拼成界面)和**客户端 Socket**(客户端怎么拨号连服务端)。所以今天先补这两块(下面"零基础速记" + 实操 0 的最小 GUI),再用 SOLO 生成**客户端网络层 + 登录窗**(含服务器地址框),接着做**读者端**(查书 / 详情 / 借书 / 我的借阅 / 还书),每加一块就起服务端 + 起客户端**联调**一次。最后做一件最重要的事——**审 AI 生成的借书代码**:它是不是把"减库存 + 写借阅记录"放进了同一个事务、减库存是不是用 `WHERE stock>0` 按影响行数防超借。

一个关键分工:客户端的 Swing 界面、Socket 收发,**由 AI 施工**;你负责**看懂它、联调把它跑通、审查关键的借书事务**。今天你会第一次看到"客户端和服务端合体":你点登录窗的按钮,请求真的过 Socket 到了服务端、查了库、结果回到界面上。

今天分五步走(外加一段审查):

步骤	你要做的事	产出
实操 0	让 Trae 写一个最小 Swing GUI 并跑通	一个窗口 + 一个按钮 + 点了弹提示,亲眼看窗口和事件
实操 1	SOLO 生成网络层 SocketClient + 登录窗(含服务器地址框)	第一次 C/S 联调:界面连上服务端、能登录分流
实操 2	SOLO 做读者端:图书查询 + 详情 + 借书	能查书、双击看详情、借书(库存实时 -1)
实操 3	SOLO 做我的借阅 + 还书	读者端一条龙跑通(查 / 借 / 看 / 还)
实操 4 ⭐	审查借书事务(信任但验证)	对照 PRD 五条清单判断"AI 写对了吗"
实操 5 ⭐	续借规则(RENEW):贴真码 + 审出漏判次数上限	借→续借(到期日 +30)→ 再续借被拒
实操 6 ⭐	逾期罚款(prompt 模式):还书算逾期天数 × 0.5 写 t_fine	造一条逾期记录还书,看到罚款金额

> 💡 先记住三个 **C/S 心法**:① 客户端**不直连数据库**,只通过网络层(SocketClient)"发一行、收一行"连服务端;② 网络请求是耗时的,必须放 **SwingWorker** 子线程,否则界面卡死;③ 先做"网络层 + 登录窗"把界面连上服务端,通了之后查书 / 借书 / 还书都是**复制这条链路换业务**——别先堆界面。

零基础速记(动手前先看懂)

速记 A · Swing 图形界面:桌面界面的"积木"

你之前写的程序是"在黑控制台打字";今天第一次做"有窗口能点鼠标"的程序。Java 用 **Swing**(`javax.swing`,自带免装)做图形界面,窗口、按钮、表格都是现成"积木"拿来拼。

- **控件积木**(类比:窗口是一块板子,控件是钉在板子上的零件):`JFrame`(大板子=顶层窗口)、`JLabel`(说明纸条)、`JTextField`(填空格=输入框)、`JButton`(按钮)、`JTable`(表格纸)、`JTabbedPane`(页签)、`JDialog`(弹出的小板子)、`JOptionPane`(弹个小纸条=提示框)。
- **布局管理器**(控件怎么摆):别用绝对坐标 `setBounds(x,y,w,h)`(一缩放就乱),用布局管理器让 Swing 自动算位置、自适应——`BorderLayout`(东西南北中)、`FlowLayout`(一排流式)、`GridBagLayout`(表单对齐)。PRD 已写死"用布局管理器"。
- **事件与回调**(点了按钮为什么会执行代码):点按钮、双击行叫**事件**;你把一段代码用 `addActionListener` "挂"到按钮上,平时不跑、**点了才被触发**——这叫**回调**(像门铃:提前设好"响了播音乐",有人按才响)。这就是**事件驱动**:程序停着等你操作,点哪个按钮跑哪段回调。
- **JTable + DefaultTableModel**(一批数据显示成表格):`JTable` 负责画表,数据由 `DefaultTableModel`(数据罐子)装着——往罐子装数据,表格自动显示。
- **SwingWorker**(网络请求为什么不能直接发):Swing 所有界面更新都在同一个线程(EDT)上;网络请求耗时,直接在点击回调里发会让 EDT 干等、**界面卡死**。解法是把请求放 `SwingWorker` 子线程,跑完回 EDT 更新界面。答案版封装成一行 `Async.run(父组件, 后台任务, 成功回调)`。

速记 B · 客户端 Socket:服务端的镜像

单元② 你学过服务端那半边(`ServerSocket` 守着等电话、`accept` 接)。客户端就是**拨号方**,是服务端的镜像:

	服务端(单元②)	客户端(今天)
建连接	`new ServerSocket(9000)` + `accept()`	`new Socket("127.0.0.1", 9000)` 拨号
发一行 / 收一行	`write+"\n"+flush` / `readLine()`	一模一样
防中文乱码	读写流指定 UTF-8	读写流指定 UTF-8(两头都要,少一头就乱码)

- **心智模型不变**:一条 socket 连接就是**一根能读一行、写一行的管子**,客户端手里也是这根管子。
- **客户端 net 三层**:`client/ui`(界面)→ `client/net/Api`(拼请求/解析响应)→ `client/net/SocketClient`(真正的收发)。客户端**不直连数据库**,只通过 SocketClient 和服务端说话。
- **SocketClient 三个关键设计**:① 长连接(连一次一直用);② `synchronized`(一次一个请求,防响应串行错位);③ 地址可配置 `configure(host,port)`(不写死,单机 127.0.0.1 / 局域网填服务器 IP,登录窗"服务器地址框"填给它)。

实操 0 · 让 Trae 写一个最小 Swing GUI 并跑通

在做真正的登录窗、读者主窗之前,先让 AI 写一个**最小的 Swing GUI 并亲手跑通**,把"速记 A"讲的窗口、按钮、事件回调落到能看见的窗口上。跑通了,你对"Swing 界面怎么长出来、事件怎么触发"就有实感了。

步骤 0.1 · 让 Trae 生成最小 GUI

【操作】

- 在 Trae SOLO 对话框,**原样复制**下面这段输入并回车:

```
用纯 Java 11 Swing 写一个最小的图形界面 demo,一个可直接运行的类 HelloSwing:
- 一个 JFrame 窗口,标题"我的第一个窗口",大小 320x200,居中显示,点关闭按钮退出程序;
- 窗口中间放一个 JButton,按钮上写"点我";
- 给按钮加点击事件(addActionListener):点了弹出一个 JOptionPane 提示框,显示一句话 + 当前时间;
- 布局用布局管理器(BorderLayout),不要写死坐标。
只要这一个类,能直接编译运行。
```

【✅ 怎么算做好】

- Trae 生成了一个 `HelloSwing.java`,内容和下面的对照基本一致。

【预期产出长什么样(逐行看懂)】

```
import javax.swing.*;    // Swing 的窗口、按钮、弹窗都在这个包
import java.awt.*;       // BorderLayout 等布局在这个包

public class HelloSwing {
    public static void main(String[] args) {
        JFrame frame = new JFrame("我的第一个窗口");          // ① 造窗口
        frame.setSize(320, 200);
        frame.setLocationRelativeTo(null);                    // 居中
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 点关闭就退出

        JButton button = new JButton("点我");                 // ② 造按钮
        button.addActionListener(e -> {                       // ③ 挂事件回调(点了
才执行)
            JOptionPane.showMessageDialog(frame,
                    "你点了按钮!现在是:" + new java.util.Date());
        });

        frame.add(button, BorderLayout.CENTER);    // ④ 用布局放到"中"区
        frame.setVisible(true);                    // 显示窗口(不写这句什么都看不到)
    }
}
```

步骤 0.2 · 跑通 + 让 AI 逐行解释

【操作】

- 直接运行 `HelloSwing` 的 `main`,屏幕正中会弹出小窗口;**点一下"点我"按钮**。
- 看不懂就让 AI 当老师,SOLO 输入:

> 用中文逐行解释 HelloSwing:JFrame 是什么,setVisible(true) 为什么必须写,addActionListener 里的代码什么时候才会执行(什么是事件和回调),BorderLayout.CENTER 是什么意思。

【✅ 怎么算做好(实操 0 的验收点)】

- 屏幕正中出现一个小窗口,中间一个"点我"按钮;点一下弹出提示框"你点了按钮!现在是:...(当前时间)"。**你亲眼看到了一个 Swing 窗口长什么样、一次事件(点按钮)怎么触发回调(弹提示)。** 带着这个实感,下面去做真正的图书馆客户端——登录窗、读者主窗骨架都是这个最小 GUI,只是积木更多。

【卡住怎么办】

- **窗口没出来 / 一闪而过**:多半是漏了 `setVisible(true)`,或程序结构不对。回一句:"HelloSwing 运行后没看到窗口,请检查是否调用了 frame.setVisible(true),并确保 main 里创建窗口后没有直接结束。只改这一处。"
- **中文标题/提示乱码**:让 AI 用系统默认编码或确认源码文件是 UTF-8 保存;一般 Trae 生成的就是对的。

实操 1 · SOLO 生成网络层 + 登录窗(含服务器地址框)

客户端分三层看(从上到下):**界面层** `client/ui` → **接口封装** `client/net/Api`(拼请求、解析响应) → **网络层** `client/net/SocketClient`(真正的收发)。今天第一步就是把最底层的 SocketClient 和最外层的登录窗做出来,让界面第一次连上服务端。

步骤 1.1 · 让 Trae 生成网络层 + 登录窗

【操作】

- 先确认单元② 的**服务端已经启动**(它在监听 9000)。
- 在 Trae SOLO 对话框,**原样复制**下面这段输入并回车:

```
帮我创建 Java Swing 客户端:
- client/net/SocketClient:与服务端 127.0.0.1:9000 建长连接,提供 request(String) 发一行收一行(synchronized);host/port 做成可配置 configure(host,port)+getHost/getPort,不要写死。
- client/ui/LoginFrame:登录窗,顶部有"服务器地址""端口"两个输入框(默认 127.0.0.1 / 9000),下面用户名、密码;登录按钮点了先按输入框 configure 再发 LOGIN,网络请求放 SwingWorker,失败弹 JOptionPane;成功按 role 打开读者主窗或管理员主窗。布局用布局管理器。
先做网络层 + 登录窗,主窗先放空壳。
```

【✅ 怎么算做好】

- Trae 生成了 `client/net/SocketClient.java`、`client/ui/LoginFrame.java`(还有空壳的 `ReaderMainFrame` / `AdminMainFrame`)。登录窗顶部有"服务器地址""端口"两个输入框,下面有用户名、密码和登录按钮。

【预期产出长什么样(对照答案版)】

- **SocketClient**——地址可配置 + `synchronized` 发一行收一行:

```java
public final class SocketClient {

    private static String host = "127.0.0.1";
    private static int port = 9000;

    /** 允许启动前(或换服务器时)自定义服务端地址。 */
    public static void configure(String h, int p) { host = h; port = p; }
    public static String getHost() { return host; }
    public static int getPort() { return port; }

    /** 发送一行请求,返回一行响应。线程安全。 */
    public static synchronized String request(String line) throws IOException {
        connect();
        out.write(line);
        out.write("\n");
        out.flush();
        String resp = in.readLine();
        if (resp == null) { close(); throw new IOException("服务器已关闭连接"); }
        return resp;
```

```
    }
}
```

> 建立连接时,读写流两头都要 UTF-8(否则中文乱码): `new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8)`、`new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8)`。

- **LoginFrame** 的登录动作——先按输入框 `configure`,再把网络请求放后台线程,成功按 role 分流:

```
private void doLogin() {
    String username = usernameField.getText().trim();
    String password = new String(passwordField.getPassword());
    // 应用服务器地址(局域网时填服务器电脑 IP)
    String host = serverField.getText().trim();
    int port = Integer.parseInt(portField.getText().trim());
    if (!host.equals(SocketClient.getHost()) || port != SocketClient.getPort()) {
        SocketClient.configure(host, port);
        SocketClient.close(); // 换了服务器,断开旧连接,下次按新地址重连
    }
    // 网络请求放子线程(SwingWorker),避免卡界面
    Async.run(this, () -> Api.login(username, password), user -> {
        Session.set(user.getId(), user.getRole(), user.getRealName());
        if (Session.isAdmin()) new AdminMainFrame().setVisible(true);
        else                   new ReaderMainFrame().setVisible(true);
        dispose(); // 关闭登录窗
    });
}
```

> 说明:答案版把 SwingWorker 封装成一行 `Async.run(父组件, 后台任务, 成功回调)`——后台线程发 `Api.login`,成功回到界面线程按 role 打开对应主窗、失败自动弹 `JOptionPane`。如果 Trae 直接用了 `SwingWorker` 也是对的,原理一样。

【卡住怎么办】

- **Trae 把 SocketClient 的地址写死了**(没有 `configure`)→ 回一句:"SocketClient 的 host/port 不要写死,做成 configure(host,port) 可配置,并提供 getHost/getPort。只改 SocketClient。"
- **登录窗没有服务器地址框**:回一句:"登录窗顶部要加'服务器地址'和'端口'两个输入框,默认 127.0.0.1 和 9000,登录时先用它们 configure。只改登录窗。"

步骤 1.2 · 第一次 C/S 联调

【操作】

- 确认单元② 的**服务端还在运行**(监听 9000)。
- 运行客户端(`LoginFrame` 的 `main`)。
- 登录窗里填:服务器地址 `127.0.0.1`、端口 `9000`、用户名 `admin`、密码 `123456`,点"登录"。

【✅ 怎么算做好(实操 1 的验收点)】

- **登录后进入管理员主窗**(现在是空壳也算通)。到这一步,你的**界面第一次真正连上了服务端**——请求过 Socket 到了服务端、查了库、结果回来分流打开了主窗。这就是第一次 C/S 联调。

【卡住怎么办(三类常见问题)】

- **连不上 / 报"Connection refused"**:
  - ① 服务端没起 → 先把单元② 的服务端跑起来;
  - ② 端口不对 → 服务端监听 9000、登录窗端口也要填 9000;
  - ③ 地址填错 → 单机就填 `127.0.0.1`(局域网才填服务器电脑的 IP)。
- **点登录后界面卡死 / 一直转圈**:网络请求没放 SwingWorker(直接在点击事件里发了)。回一句:

```
登录的网络请求要放在 SwingWorker 的 doInBackground 里(或用 Async 封装),不要直接在按钮点击事件里发,否则会卡界面。只改这一处。
```

- **能登录但中文乱码**(比如返回的真实姓名是乱码):客户端读写流没用 UTF-8。回一句:

```
SocketClient 的收发流要用 UTF-8:InputStreamReader 和 OutputStreamWriter 都指定 StandardCharsets.UTF_8。只改这一处。
```

实操 2 · 读者端:图书查询 + 详情 + 借书

登录通了,做读者端核心。**分两小步**:先查询和详情(纯读,简单),再借书(涉及事务,是实操 4 的审查重点)。

步骤 2.1 · 图书查询 + 详情

【操作】

- 在 SOLO 里承接上文,**原样输入**:

```
基于现有代码,做读者端:
- 服务端加 BOOK_QUERY|关键字|分类id(BookService/BookDao,LEFT JOIN 分类带出 categoryName,Json 编码返回)、CATEGORY_LIST;
- 客户端读者主窗:搜索栏(关键词+分类下拉)+ JTable 图书列表;双击一行弹图书详情对话框,详情里有"借书"按钮。
先做查询和详情展示,借书下一步。只加图书查询相关。
```

【✅ 怎么算做好】

- Trae 生成了服务端 `BookService` / `BookDao`(`BOOK_QUERY`、`CATEGORY_LIST`),客户端 `ReaderMainFrame`(选项卡)、`BookQueryPanel`(JTable + 搜索栏)、`BookDetailDialog`。

【预期产出长什么样(对照答案版)】

- 读者主窗用选项卡装两个面板:

```
JTabbedPane tabs = new JTabbedPane();
tabs.addTab("图书查询", new BookQueryPanel());
tabs.addTab("我的借阅", new MyBorrowPanel());  // 我的借阅在实操 3 加
setContentPane(tabs);
```

- `BookQueryPanel` 用 `DefaultTableModel` 装 JTable 数据,查询放后台线程:

```
private final DefaultTableModel model = new DefaultTableModel(
        new Object[]{"ID", "书名", "作者", "出版社", "分类", "库存"}, 0) {
    @Override public boolean isCellEditable(int r, int c) { return false; }
};
private final JTable table = new JTable(model);

/** 查询图书并刷新表格。public 方便借书成功后外部刷新。 */
public void query() {
    String keyword = keywordField.getText().trim();
    Category cat = (Category) categoryBox.getSelectedItem();
    int categoryId = cat == null ? 0 : cat.getId();
    Async.run(this, () -> Api.queryBooks(keyword, categoryId), this::fill);
}
```

> 服务端 `BookDao.query` 用 `LEFT JOIN t_category` 把分类名带出来,查询用 `PreparedStatement` 参数化(`title LIKE ? OR author LIKE ?`)。这是**纯读操作,属于 🟢 高信任档**——扫一眼参数化到位、能跑通即可,不用逐行抠。

【✅ 联调验证】

- 起服务端 → 起客户端 → 登录读者 `zhangsan/123456` → 进读者主窗 → 图书查询页看到种子图书列表(带分类名、库存);输关键字点查询能筛选;双击一行弹详情对话框。**中文书名不乱码**(乱码回实操 1 的 UTF-8 检查)。

【卡住怎么办】

- **列表空但服务端有数据**:服务端 `BOOK_QUERY` 没返回、或客户端没解析对。让 AI 解释:"用中文说明 BOOK_QUERY 从客户端发出到服务端返回、再到 JTable 显示的完整链路,帮我定位哪一环没通。"
- **分类那一列是空的**:服务端查询没 `LEFT JOIN` 带出分类名。回一句:"BOOK_QUERY 的 SQL 要 LEFT JOIN t_category 带出 categoryName 一起返回。只改这一处。"

步骤 2.2 · 借书(先生成,实操 4 专门审)

【操作】

- 在 SOLO 里**原样输入**:

```
基于现有代码,加借书:
- 协议 BORROW|userId|bookId;服务端 BorrowService.borrow 在一个事务里:UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0(按影响行数判库存)、校验读者未还数<max_borrow、插入 t_borrow(due=今+30天),成功 commit 返 OK|应还日期,失败 rollback 返 ERR。
- 客户端详情对话框"借书"按钮调 BORROW,提示结果并刷新库存。只做借书。
```

【✅ 怎么算做好】

- Trae 生成了服务端 `BorrowService.borrow`(事务)和客户端 `BookDetailDialog` 的借书按钮。

【预期产出长什么样(对照答案版)】

- 客户端借书按钮——把请求放后台,成功后标记 borrowed 让列表刷新:

```
private void doBorrow() {
    Async.run(this, () -> Api.borrow(Session.userId(), book.getId()), due -> {
        borrowed = true;
        JOptionPane.showMessageDialog(this,
                "借阅成功!应还日期:" + due, "成功",
JOptionPane.INFORMATION_MESSAGE);
        dispose();
    });
}
```

> 借书成功后,`BookQueryPanel` 会检查 `dlg.isBorrowed()`,为真就重新 `query()` 刷新列表——所以你会看到**库存实时减 1**。

> ⚠️ 服务端 `BorrowService.borrow` 的代码**先不急着看**,留到实操 4 让你亲自审。这一步先把它生成出来、联调跑通。

【✅ 联调验证】

- 详情对话框点"借这本书" → 提示"借阅成功!应还日期:..." → 关闭后图书列表该书**库存 -1**。再借,库存继续减;库存到 0 时详情里借书按钮变"暂无库存"。

【卡住怎么办】

- **借书提示成功但列表库存没变**:借成功后没刷新列表。回一句:"借书成功后要重新调 query() 刷新图书列表的库存。只改这一处。"
- **借书报服务端异常**:多半是服务端 `borrow` 里 SQL 或事务写错——先联调跑通,具体对不对留到实操 4 审。

实操 3 · 我的借阅 + 还书

【操作】

- 在 SOLO 里**原样输入**:

```
基于现有代码,加"我的借阅":
- 协议 MY_BORROW|userId(BorrowDao LEFT JOIN 出书名/借出/应还/状态)、
RETURN|borrowId(事务:记录标 returned+库存+1,防重复归还)、RENEW|borrowId;
- 客户端读者主窗加"我的借阅"选项卡:JTable 显示我的借阅 + 每行"还书/续借"按钮。只做借阅记录和还书。
```

【✅ 怎么算做好】

- Trae 生成了服务端 `BorrowService.returnBook`(事务)/ `renew`、`BorrowDao.findByUser`,客户端 `MyBorrowPanel`(我的借阅选项卡 + 还书 / 续借按钮)。

【预期产出长什么样(对照答案版)】

- 客户端还书——选中未还记录 → 后台请求 → 刷新:

```
private void doReturn() {
    int id = selectedBorrowId();
    if (id < 0) return;
    Async.run(this, () -> { Api.returnBook(id); return null; }, v -> {
        JOptionPane.showMessageDialog(this, "还书成功!", "成功",
JOptionPane.INFORMATION_MESSAGE);
        load();
    });
}
```

- 服务端 `returnBook` 同样用事务(改记录状态 + 库存 +1,同生共死),并且**防重复归还**——只有当记录当前是 `borrowed` 才置为 `returned`:

```
// 1. 仅当记录当前处于 borrowed 状态时才置为 returned
String updBorrow = "UPDATE t_borrow SET status = 'returned', return_time = NOW()
"
                 + "WHERE id = ? AND status = 'borrowed'";
// executeUpdate,affected == 0 说明该记录不存在或已归还 → rollback + 报错
// 2. 找到对应 book_id;3. UPDATE t_book SET stock = stock + 1 WHERE id = ?;commit
```

> 这里和借书是**同一个套路**:`WHERE id=? AND status='borrowed'` 按影响行数判断——影响行数为 0 就说明"已经还过了",防止重复归还把库存越加越多。

【✅ 联调验证(读者端一条龙)】

- 登录 `zhangsan` → 查书 → 借一本(库存 -1)→ 切到"我的借阅"看到这条记录(状态"借阅中")→ 选中点"还书"(提示成功)→ 记录变"已归还",回图书查询看该书**库存 +1 回来。读者端完整跑通。**

【卡住怎么办】

- **还书后没刷新 / 状态没变**:还书成功后没重新 `load()`。回一句:"还书成功后要重新 load() 刷新我的借阅列表。只改这一处。"
- **同一条记录能还两次、库存越加越多**:服务端没防重复归还。回一句:"还书的 UPDATE 要带 status='borrowed' 条件,按影响行数判断,影响行数为 0 就报'已归还'并 rollback,不能再给库存 +1。只改还书这一处。"

实操 4 · ⭐ 借书事务代码审查(信任但验证)

系统跑通了,但"跑通 ≠ 正确"——尤其借书涉及并发,**单机一个人点测不出问题**。现在拿单元① 写进 PRD 的规则,审 AI 生成的 `BorrowService.borrow`。这是今天最重要的一环:**AI 施工,你验收。**

步骤 4.1 · 打开借书代码,对照答案版

【操作】

- 打开 Trae 生成的 `server/service/BorrowService.java`,找到 `borrow` 方法。
- 下面是答案版**正确**的 borrow(作为审查对照标准):

```
public String borrow(int userId, int bookId) throws SQLException {
```

```
    Connection conn = null;
    try {
        conn = DBUtil.getConnection();
        conn.setAutoCommit(false); // 开启事务

        // 1. 检查该读者"未还数量"是否已达其借阅上限
        int maxBorrow = queryMaxBorrow(conn, userId);
        int unreturned = countUnreturned(conn, userId);
        if (unreturned >= maxBorrow) {
            conn.rollback();
            throw new IllegalStateException("已达借阅上限(" + maxBorrow + " 本),请先
还书");
        }

        // 2. 扣库存:带 stock > 0 条件,按影响行数判断是否成功(防超借核心)
        int affected;
        String updSql = "UPDATE t_book SET stock = stock - 1 WHERE id = ? AND
stock > 0";
        try (PreparedStatement ps = conn.prepareStatement(updSql)) {
            ps.setInt(1, bookId);
            affected = ps.executeUpdate();
        }
        if (affected == 0) {
            conn.rollback();
            throw new IllegalStateException("库存不足,借阅失败");
        }

        // 3. 插入借阅记录:borrow_time = 现在,due_time = 现在 + 30 天
        LocalDateTime now = LocalDateTime.now();
        LocalDateTime due = now.plusDays(BORROW_DAYS);
        String insSql = "INSERT INTO t_borrow(user_id, book_id, borrow_time,
due_time, status) "
                      + "VALUES(?, ?, ?, ?, 'borrowed')";
        try (PreparedStatement ps = conn.prepareStatement(insSql)) {
            ps.setInt(1, userId);
            ps.setInt(2, bookId);
            ps.setTimestamp(3, Timestamp.valueOf(now));
            ps.setTimestamp(4, Timestamp.valueOf(due));
            ps.executeUpdate();
        }

        conn.commit(); // 三步全部成功,提交事务
        return due.toString().replace('T', ' ');
    } catch (SQLException | RuntimeException e) {
        rollbackQuietly(conn);  // 任一步失败,回滚
        throw e;
    } finally {
        restoreAndClose(conn);  // 恢复 autoCommit 并关连接
    }
}
```

【看不懂就问】

- 让 Trae 用大白话解释:"用中文逐行解释这个 borrow 方法:为什么要 setAutoCommit(false) 开事务、为什么减库存用 WHERE stock>0、为什么要看 executeUpdate 的返回值(影响行数)。"

步骤 4.2 · ⭐ 审查清单:逐条判断"AI 写对了吗"

【操作】

- 对着 AI 刚生成的 borrow,拿下面这张清单一条条量(这就是"拿 PRD 验代码"):

#	审查点(PRD 要求的)	在答案版里怎么体现	AI 写对了吗
1	**是不是一个事务?**	`setAutoCommit(false)` 开事务,三步全成才 `commit()`,任一步失败 `rollback()`	有这三件套 = 对
2	**减库存是不是 `WHERE stock>0`?**	`UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0`	有 `AND stock>0` = 对
3	**是不是按影响行数判库存?**	`affected = ps.executeUpdate(); if (affected == 0) { rollback; 报"库存不足" }`	用影响行数判、不是先 SELECT = 对
4	**减库存和写记录在不在同一事务?**	扣库存 UPDATE 和 插入 t_borrow 用**同一个 `conn`**,一起 commit	同一 conn / 一起提交 = 对
5	**借阅上限校验了吗?**	借前 `countUnreturned >= maxBorrow` 就 rollback	有上限校验 = 对(🟡 中信任)

【✅ 怎么算做好】

- **五条都对** = AI 这次借书写对了,信任但已验证,放行。你**心里清楚它对在哪、为什么对**。

步骤 4.3 · AI 常见的三种写错(认出来 + 让它改)

【要点】如果 AI 生成的 borrow 是下面这些样子,就是**埋了坑**——注意:**单机一个人点,这三种都能借成功、库存也减了,表面全对**,坑藏在并发里。用纠错三步法让它改:

- ❌ **写法一:先查后减(check-then-act)**。先 `SELECT stock`,Java 里 `if (stock > 0)`,再 `UPDATE stock = stock - 1`(不带 `WHERE stock>0`)。
  - 后果:并发下两个人同时查到还剩 1,都判断"够",都减,库存变 -1(**超借**)。
  - 纠错:

    ```
    借书扣库存不要先 SELECT 再 UPDATE,改成 UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0,按影响行数是否为 0 判断库存不足。只改这一处。
    ```

- ❌ **写法二:减库存和写记录不在一个事务**。分两次拿连接、各自自动提交。
  - 后果:可能减了库存却没写成记录(中间报错),数据对不上。

- 纠错:

```
借书的扣库存和插入 t_borrow 必须在同一个事务里:同一个 Connection、setAutoCommit(false),两步都成功才 commit,任一失败 rollback。只改 borrow 这一处。
```

- ❌**写法三:UPDATE 了但没看影响行数**。带了 `WHERE stock>0`,但不判 `affected == 0`,不管有没有减成都往下写记录。
  - 后果:库存为 0 时 UPDATE 没减到任何行,却照样写了借阅记录——**凭空多借一本**。
  - 纠错:

```
扣库存的 UPDATE 后要判断 executeUpdate 的返回值,等于 0 说明没库存,要 rollback 并返回"库存不足",不能继续写借阅记录。只改这一处。
```

【✅ 怎么算做好(实操 4 的验收点)】

- 你**对照 PRD 五条清单逐条判断**过 AI 生成的借书代码,认得出"先查后减 / 不在一个事务 / 不判影响行数"这三种埋坑写法;若 AI 写错了,用纠错三步法让它改对。**到这一步,你手里多了一双"审 AI 关键代码的眼睛"。**

> 🔬 **信任但验证**:AI 写借书又快又能跑,但**对不对要你判断**。单机测不出的并发坑,靠的就是你拿 PRD 一条条量。这就是"跑通≠正确",这就是为什么借书是 🔴 必人工的代码。**AI 施工,你验收——拍板的是你。**

实操 5 · ⭐ 续借规则(RENEW):贴真码 + 亲手审出"漏判次数上限"

实操 3 里你让 AI 顺手生成了"续借按钮",点了能把应还日期往后推。但**"能点"不等于"规则对"**:续借该延几天?已还的书能不能续?**同一本书能无限续下去吗?** 这些是业务规则。这一步就拿"续借"再练一次**信任但验证**——先贴真码对照,再拿规则清单一条条量,**你会亲手审出连答案版都漏了的一条规则(续借次数上限)**,然后用迭代把它补上。

步骤 5.1 · 完善续借规则(生成 + 贴真码对照)

【操作】

- 在 SOLO 里**原样输入**:

```
基于现有代码,完善读者端"续借"(RENEW)的服务端规则:
- 协议 RENEW|borrowId;
- BorrowService.renew:只有当这条借阅记录当前是 borrowed 状态才允许续借,把 due_time 用
  UPDATE ... SET due_time = DATE_ADD(due_time, INTERVAL 30 DAY) WHERE id=? AND status='borrowed'
  往后延 30 天,按影响行数判断:影响行数为 0 说明记录不存在或已归还,返回 ERR(不能续借);
- 续借成功后 SELECT 出新的 due_time,以 OK|新应还日期 返回;
- 客户端"我的借阅"里"续借"按钮调 RENEW,成功弹新应还日期并刷新列表。
只改续借相关,别动借书/还书。
```

【✅ 怎么算做好】

- Trae 生成/完善了服务端 `BorrowService.renew`,客户端"我的借阅"续借按钮走 `Api.renew`。

【预期产出长什么样(对照答案版真码)】

- 服务端 `renew`——常量 `RENEW_DAYS = 30`,用 `status='borrowed'` 守卫 + 按影响行数判"已还不能续":

```
private static final int RENEW_DAYS = 30; // 续借延长 30 天

public String renew(int borrowId) throws SQLException {
    String sql = "UPDATE t_borrow SET due_time = DATE_ADD(due_time, INTERVAL ? DAY) "
               + "WHERE id = ? AND status = 'borrowed'";
    try (Connection conn = DBUtil.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, RENEW_DAYS);
        ps.setInt(2, borrowId);
        int affected = ps.executeUpdate();
        if (affected == 0) {                          // 影响行数为 0 = 记录不存在或已归还
            throw new IllegalStateException("该记录不存在或已归还,无法续借");
        }
    }
    // 查询续借后的新应还日期返回(SELECT due_time WHERE id=?)
    ...
}
```

- 客户端续借按钮(和还书同一套 `Async` 后台请求套路):

```
private void doRenew() {
    int id = selectedBorrowId();
    if (id < 0) return;
    Async.run(this, () -> Api.renew(id), newDue -> {
        JOptionPane.showMessageDialog(this, "续借成功!新应还日期:" + newDue, "成功", JOptionPane.INFORMATION_MESSAGE);
        load();
    });
}
```

步骤 5.2 · ⭐ 规则审查:拿清单量,审出答案版漏的那一条

【操作】

- 对着续借代码,拿下面这张清单一条条量。**注意第 4 条**:

#	续借规则	在答案版里怎么体现	写对了吗
1	**续借延多少天?**	`RENEW_DAYS = 30`,`DATE_ADD(due_time, INTERVAL 30 DAY)` ——从旧应还日往后加,不是从今天算	有常量、从 due_time 延 = 对
2	**已归还的能续吗?**	`WHERE id=? AND status='borrowed'`,`returned` 的记录匹配不到	有 status 守卫 = 对
3	**怎么知道"续失败"?**	`affected == 0` 就返回 ERR,不是都回 OK	用影响行数判 = 对
4	🔴 **同一本书能无限续吗?(续借次数上限)**	**——答案版这段没有次数上限!任何 borrowed 记录都能一续再续**	❌ **漏了**

> 💡 第 4 条不是我给错了代码——是**故意让你练**:审查不是照着标准答案打钩,是**拿业务规则去照,连"看着对"的代码也要照出它没覆盖的边界**。真实项目里,AI(和人)最常漏的就是这种"没人明说、但业务上一定有"的上限。这条下面用迭代补。

步骤 5.3 · ⭐ 迭代补"续借次数上限":故意不给字段提示,审 AI 漏没漏

【操作】

- 第一句**故意"轻描淡写"**地提需求(**不告诉它该怎么存续借次数**),看 AI 会不会漏判、会不会自作主张。SOLO 输入:

```
给续借加一条规则:每本书最多只能续借 1 次,已经续借过的再点续借要返回"已达续借次数上限,无法续借"。
```

【观察 AI 怎么处理(这是审查刻意练习)】

续借次数得**有地方记**——`t_borrow` 表原本没有"续借次数"这一列。AI 常见三种反应,都要你审:

- 它**加了字段**(`t_borrow` 加 `renew_count INT DEFAULT 0`),续借时 `WHERE ... AND renew_count < 1` 且 `SET renew_count = renew_count + 1`——✅ 对,但要看它**改没改建表 SQL**、老数据默认 0 没。
- 它**没加字段 / 干脆漏判**(嘴上说加了、代码里没拦)——❌ 让它落到字段上。
- 它**加了判断但没自增**(判了 `renew_count < 1`,却忘了 `+1`)——❌ **这就是"看着对、实际拦不住":第一次没 +1,第二次照样能续**。最隐蔽,必须验证到"第二次被拒"才算数。

【补齐后的正确形态(让 AI 对到这里)】

```
ALTER TABLE t_borrow ADD COLUMN renew_count INT NOT NULL DEFAULT 0 COMMENT '已续借次数';
```

```
// 一句 UPDATE 原子完成"限次数 + 记一笔",仍按影响行数判:
String sql = "UPDATE t_borrow "
           + "SET due_time = DATE_ADD(due_time, INTERVAL ? DAY), renew_count =
renew_count + 1 "
           + "WHERE id = ? AND status = 'borrowed' AND renew_count < 1";
// affected == 0 → 记录不存在 / 已归还 / 已达上限,统一返回 ERR
```

> 💡 这和借书 `WHERE stock>0` 按影响行数防超借**是同一套心法**:把并发下不安全的"先查再改"压成一句带条件的 UPDATE。要是看到 AI 先 `SELECT renew_count` 再 Java 里 `if`,提醒它并成一句:`续借的次数判断和自增合并成一句 UPDATE ... WHERE renew_count < 1,按影响行数判断,不要先 SELECT 再判断。只改这一处。`

【✅ 怎么算做好(实操 5 的验收点)—— 必须验到"第二次被拒"】

1. 借一本 → 到"我的借阅"续借一次 → 提示成功、应还日期 **+30 天**;
2. 对同一条记录**再点一次续借 → 这次必须被拒**,提示"已达续借次数上限"。**只验到第 1 步不算通过——次数上限的坑,恰恰藏在第二次。**

【卡住怎么办】

- **续借后应还日期没变**:`借书成功后要重新 load() 刷新列表,续借的 due_time 要从旧应还日 DATE_ADD 30 天。只改续借这一处。`
- **续了第二次还能成功**:多半是判了次数没自增。`续借要在同一句 UPDATE 里 SET renew_count = renew_count + 1,并 WHERE renew_count < 1 按影响行数判断。只改这一处。`

实操 6 · ⭐ 逾期罚款(prompt 模式:demo 无真码,强 prompt + 行为审查)

`sql/init.sql` 里早就建好了一张 `t_fine`(罚款表),但到现在**一条罚款都没算过**——表建了、逻辑空着。这一步补上"还书时算逾期罚款"。**和续借不一样:答案版工程没有现成的罚款计算真码**,所以这一步**走 prompt 模式**——给你一段**强 prompt**(把规则说死)、一份**行为描述**,重点在**审**:日期差怎么算、没逾期该不该罚、金额精度会不会翻车。这正好复用实操 4 借书事务那套"信任但验证"。

先看清 t_fine 字段再让 AI 写:

```
CREATE TABLE t_fine (
  id         INT PRIMARY KEY AUTO_INCREMENT,
  borrow_id INT            NOT NULL COMMENT '关联的借阅记录',
  amount     DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '罚款金额',  -- ⚠️ DECIMAL
定点,不是 double
  paid       TINYINT       NOT NULL DEFAULT 0 COMMENT '0 未缴 / 1 已缴',
  CONSTRAINT fk_fine_borrow FOREIGN KEY (borrow_id) REFERENCES t_borrow(id)
);
```

步骤 6.1 · 用强 prompt 生成逾期罚款

【操作】

- 在 SOLO 里**原样输入**(这段 prompt 把规则说死了,照抄别删):

```
给还书流程加"逾期罚款"。规则说死如下,严格照做:
- 罚款标准:每逾期 1 天罚 0.5 元;逾期天数 = 实际归还时间与应还时间 due_time 的天数差,向上取整;
- 只在"实际归还时间 晚于 due_time"时才罚;没逾期(按时或提前还)不写任何罚款记录;
- 计算和写入放进还书那个事务里:还书 UPDATE 成功之后、同一个 Connection、一起 commit;
  逾期就 INSERT INTO t_fine(borrow_id, amount, paid) VALUES(?, ?, 0);
- 金额一律用 BigDecimal / DECIMAL 定点计算,禁止用 double/float 算钱;
- 还书返回值带上本次罚款金额(没逾期就是 0),客户端还书成功时把罚款金额提示给用户。
只改还书相关,别动借书。
```

【✅ 怎么算做好】

- Trae 在 `returnBook` 事务里补出了"算逾期天数 → 逾期才写 t_fine"的逻辑,客户端还书提示里带出罚款金额。

【预期产出的"行为描述"(不是答案版真码,以你审查 AI 生成的为准)】

```
// 参考思路(非答案版真码,AI 生成后以你审查为准):
// 1. 在还书事务里、UPDATE 借阅记录成功之后,取出这条记录的 due_time;
// 2. long overdueDays = 逾期天数(实际归还时间 与 due_time 的天数差,向上取整);
// 3. if (overdueDays > 0) {                          ← 没逾期(<=0)就跳过,一条罚款都不写
//        BigDecimal amount = new
BigDecimal("0.5").multiply(BigDecimal.valueOf(overdueDays));
//        INSERT INTO t_fine(borrow_id, amount, paid) VALUES(borrowId, amount,
0);
//    }
// 4. 罚款 INSERT 和还书 UPDATE 用同一个 conn,一起 commit(还成功了罚款才算数)
```

步骤 6.2 · ⭐ 行为审查:复用借书事务那把尺子

【操作】

- 对着 AI 写的罚款代码,拿下面清单量(这就是"拿规则验代码"):

#	审查点	怎么算对	信任档
1	**日期差拿谁比谁?**	拿**实际归还时间**和**应还** **`due_time`** 比,**不是**和借出时间 `borrow_time` 比	🟡 中信任
2	**没逾期会不会误罚?**	逾期天数 ≤ 0 时**一条罚款都不写**(不是写一条 0 元)	🟡 中信任
3	**金额精度会翻车吗?**	用 `BigDecimal` / `DECIMAL` 算,**不用** **`double`**(`0.5 * 3` 浮点会算出 `1.5000000002` 脏值)	🔴 必人工(涉及钱)
4	**罚款和还书在一个事务吗?**	INSERT `t_fine` 和还书 UPDATE 用**同一个 conn**、一起 commit;还书回滚了罚款也别留	🔴 必人工
5	**天数怎么取整?**	差 1 天 10 分钟算几天?按 PRD "向上取整",不确定就问 AI 它用了 `ceil` 没	🟡 中信任

> 💡 第 3 条为什么标🔴——**只要涉及钱,一律必人工**。AI 图省事很爱写 `double amount = 0.5 * days`,单机看着没事,金额一累加浮点误差就冒出来,对账差几分钱谁都说不清。算钱用 `BigDecimal` / `DECIMAL`,铁律。

步骤 6.3 · ⭐ 让 AI 解释 + 主动制造 bug

【操作】

- 让 AI 用中文把它自己写的算法讲一遍(理解而非默写),顺手把两个最爱踩的坑试出来。SOLO 输入:

```
用中文逐行解释你写的逾期罚款:逾期天数是拿哪两个时间算的、怎么取整、金额用什么类型算、
没逾期时会不会写罚款记录。如果金额用了 double,改成 BigDecimal;如果没逾期也 INSERT 了,改成只有逾期才写。只改还书相关。
```

【踩坑表】

问题	原因	可直接复制的追问 prompt
按时还也生成了一条 0 元罚款	没判"逾期天数 > 0" 就无脑 INSERT	`逾期罚款只在实际归还时间晚于 due_time 时才写 t_fine;没逾期不要写任何罚款记录。只改这一处。`
金额出现 `1.5000000002` 脏值	用 double/float 算钱	`算罚款金额改用 BigDecimal 定点计算,0.5 元/天 × 逾期天数,不要用 double。只改这一处。`
还书回滚了、罚款却留下了	罚款 INSERT 没进还书事务	`罚款 INSERT t_fine 要和还书 UPDATE 用同一个 Connection、同一个事务,一起 commit,任一失败一起 rollback。只改还书这一处。`
逾期天数算成"从借出到现在"	拿 borrow_time 比而非 due_time	`逾期天数要用实际归还时间和应还时间 due_time 的差,不是和借出时间 borrow_time 的差。只改这一处。`

【✅ 怎么算做好(实操 6 的验收点)—— 用"造数据"验,单机也能测】

- 造一条**已逾期**记录:直接在 MySQL 里 `UPDATE t_borrow SET due_time = '2020-01-01 00:00:00' WHERE id = 你借的那条;`,然后在客户端还这本书 → 查 `t_fine` 应生成一条金额 = 逾期天数 × 0.5 的记录,客户端提示里带出罚款金额;
- 造一条**没逾期**记录:正常借一本立刻还 → `t_fine` **不应**新增任何记录。
- 两条都验到,实操 6 通过。

> 🔬 **信任但验证**:罚款没有答案版真码,但**有规则、有清单**——规则就是你的验收标准,清单就是你的尺子。这和实操 4 借书事务一个道理:碰钱、碰事务的代码,是🔴必人工,必须你逐条盯。

今天做完了什么(自检清单)

- [ ] 看懂零基础两块:Swing 图形界面(窗口/控件/布局/事件回调/表格/SwingWorker)+ 客户端 Socket(拨号连服务端/发一行收一行)
- [ ] 最小 Swing GUI 跑通:一个窗口 + 一个按钮 + 点了弹提示,亲眼看窗口和事件(实操 0)
- [ ] 第一次 C/S 联调通过:客户端网络层 + 登录窗做好,界面连上服务端、按 role 分流(实操 1)
- [ ] 读者端图书查询 + 详情跑通:查得到、筛得动、双击看详情、中文不乱码(实操 2.1)
- [ ] 借书跑通:借一本、图书列表库存实时 -1(实操 2.2)
- [ ] 我的借阅 + 还书跑通:读者端一条龙(查 → 借 → 看 → 还,库存 +1 回来)(实操 3)
- [ ] 审了借书事务:对照 PRD 五条清单判断"AI 写对了吗",认得出三种埋坑写法(实操 4)
- [ ] 续借规则:贴真码对照、审出答案版漏的"续借次数上限",迭代补齐并验到"第二次续借被拒"(实操 5)
- [ ] 逾期罚款:强 prompt 生成、拿清单审(日期差/未逾期不罚/金额精度/事务),造逾期数据验到罚款金额(实操 6)

七项都打勾,今天的目标就达成了——**客户端由 AI 施工、你把它联调跑通、并审查了最关键的借书事务。**

课后任务

1. **补一个审查练习**:让 Trae 故意用"先 SELECT 再 UPDATE"的写法重写一遍 borrow(prompt:"把 borrow 改成先 SELECT stock 判断再 UPDATE 的写法"),然后你**自己指出**这么写在并发下会超借,再用纠错三步法把它改回 `WHERE stock>0` 按影响行数的正确写法。体会"同一段功能,写法不同、并发安全天差地别"。
2. **给自己的项目想一想**:课后任务里你为自己的小型 C/S 系统写过 PRD(单元① 课后)。想一想——**你那个系统里有没有"像借书一样必须放进一个事务、要防并发"的操作**?(比如抢票的扣票、预约的占位、转账的加减。)把它列出来,标成 🔴 必人工档,提醒自己生成后要审。

> 记住今天最重要的一句话:**跑通 ≠ 正确。** 界面能点、功能能用,只是第一步;涉及事务和并发的关键代码,必须你拿 PRD 去审——这就是"信任但验证"。